

Nome: _____ Matrícula: _____

Instruções.

- São 15 questões. **Resolva todas**, na ordem em que aparecem, identificando o número de cada questão.
- Resolva à **mão**, em folha própria. Ao final, **escaneie** ou fotografe e **converta para um único PDF**, entregando no **Google Classroom**. Páginas ilegíveis serão desconsideradas.
- Escreva o seu **nome e matrícula em todas as folhas**.
- **Estilo funcional obrigatório**: use `map`, `filter`, `reduce` (de `functools`) e/ou **recursão**. **Proibido**: laços `for/while`, comprehensions, `.append`, mutação por índice, `set`, classes, POO. Violar zera a questão.
- Pode usar `from functools import reduce` e `import math` livremente.

Questão 1 — Variação do saldo da conta digital

O saldo de fechamento (em R\$) de uma conta digital nos últimos quatro dias foi:

$$\text{saldos} = [1200, 1260, 1197, 1290]$$

Escreva, em **uma única expressão** com `map`, a função `variacoes(saldos)` que retorne a lista das variações percentuais dia-a-dia (em %), arredondadas a 2 casas decimais. Saída esperada: `[5,0; -5,0; 7,77]`.

Questão 2 — PIX acima do limite noturno

Dada uma lista `pix` com os valores (em R\$) das transferências PIX feitas por um cliente durante a noite, defina `acima_do_limite(pix, limite)` usando **somente** `filter`, que devolve a lista dos valores estritamente acima de `limite` (candidatas a bloqueio antifraude).

Em seguida, defina `qtd_bloqueios(pix, limite)` usando `filter` + `reduce` (sem `len`).

Exemplo:

```
pix = [120.0, 2500.0, 89.9, 4300.0, 15.0, 1800.0]
# acima_do_limite(pix, 1500) -> [2500.0, 4300.0, 1800.0]
# qtd_bloqueios(pix, 1500)    -> 3
```

Questão 3 — Retorno acumulado de um criptoativo com reduce

Os retornos diários (em fração, ex.: $-0,034$ para $-3,4\%$) de um criptoativo em seis dias são dados em `retornos`. Defina, em **uma única expressão** usando `reduce`, a função `retorno_acumulado(retornos)` que devolve $\prod_i (1 + r_i) - 1$, isto é, o retorno equivalente do período.

```
retornos = [0.012, -0.034, 0.051, 0.008, -0.021, 0.027]
# retorno_acumulado(retornos) ~ 0.0413
```

Explique em 1–2 linhas por que **não** se pode simplesmente somar os retornos diários.

Questão 4 — Compras internacionais no cartão

Dada uma lista `compras` (valores em US\$), a cotação dolar (R\$ por US\$) e a alíquota de IOF `iof` (em fração, ex.: $0,0338$ para $3,38\%$), escreva, usando **apenas** `map`, a função `fatura_em_reais(compras, dolar, iof)` que retorne o valor de cada compra em R\$ **já com IOF** (i.e., $\text{valor} \cdot \text{dolar} \cdot (1 + \text{iof})$), arredondado a 2 casas.

Questão 5 — Aquecimento funcional

Reescreva o trecho **imperativo** abaixo, que calcula o volume total das transações **aprovadas** de uma fintech, em **uma única expressão** usando `filter` + `reduce` (sem laços, sem variáveis intermediárias):

```
def volume_aprovado(transacoes):
    total = 0
    for id_tx, valor, status in transacoes:
        if status == "APROVADA":
            total = total + valor
    return total
```

Questão 6 — Rotativo do cartão por recursão

Sem usar laços, `**` ou `math.pow`, defina **recursivamente** a função `divida_rotativo(d, i, n)` que devolve $D \cdot (1 + i)^n$, onde D é o valor não pago da fatura, i a taxa mensal do rotativo e $n \geq 0$ o número de meses sem pagamento.

- (a) Defina explicitamente o **caso-base** e o **caso recursivo**, justificando a terminação.
 - (b) Mostre que `divida_rotativo(500, 0, 10, 3) = 665,5`.
-

Questão 7 — Recursão sobre naturais: número da conta

Defina **recursivamente** (sem laços, sem `str`, sem `map/filter/reduce`):

- (a) `soma_digitos(n)` — soma dos dígitos do número de conta $n \geq 0$. *Ex.*: `soma_digitos(48271) = 22`.
- (b) `raiz_digital(n)` — aplica `soma_digitos` repetidamente **de forma recursiva**, até sobrar um único dígito (usado como dígito verificador simplificado). *Ex.*: `raiz_digital(48271) = 4`.

Justifique a terminação de `raiz_digital`.

Questão 8 — Recursão estrutural sobre o extrato

Implemente, **apenas por recursão estrutural** (com cabeça `xs[0]` e cauda `xs[1:]`; **sem** `map/filter/reduce`, **sem** laços, **sem** `len`, `sum`, `max`), sobre uma lista de valores de compras no cartão:

- (a) `maior_compra(xs)` — maior valor da lista (assuma `xs` não-vazia);
 - (b) `total_fatura(xs)` — soma da lista;
 - (c) `ticket_medio(xs)` — média aritmética, usando (b) e uma função auxiliar `tamanho(xs)` também recursiva.
-

Questão 9 — Esteira de concessão de crédito (pipeline)

Uma fintech mantém os proponentes a crédito no formato `(nome, score, renda)`:

```
clientes = [
    ("Ana", 720, 4500),
    ("Bruno", 610, 3200),
    ("Carla", 805, 7800),
    ("Diego", 690, 5100),
    ("Elisa", 750, 2900),
    ("Fabio", 580, 6400),
]
```

Implemente, **cada uma em uma única expressão** com `map/filter/reduce` (sem laços, sem comprehensions):

- (a) `aprovados(clientes)` — nomes dos clientes com `score` ≥ 700 ;
 - (b) `renda_media(clientes)` — média das rendas, usando `reduce` (sem `sum`);
 - (c) `parecer(clientes)` — lista de strings `"Ana: SCORE=720 RENDA=4500 (APROVADO)"` ou `"... (NEGADO)"`, com `map`;
 - (d) `melhor_score(clientes)` — tupla `(nome, score)` com *maior* `score` via `reduce` (sem `max`).
-

Questão 10 — Implementando `minha_reduce`

Sem usar `map`, `filter`, `reduce` ou laços, defina **por recursão** sobre listas:

```
def minha_reduce(f, xs, inicial):
    """Equivalente a functools.reduce(f, xs, inicial)."""
```

Em seguida, usando **somente** `minha_reduce` e `lambda`, escreva **em uma única linha cada**:

- (a) `saldo_final(xs)` — soma dos lançamentos em `xs` (créditos positivos, débitos negativos);
- (b) `maior_pix(xs)` — maior valor de `xs` (sem `max`; assumo `xs` não-vazia, use `xs[0]` como inicial);
- (c) `inverte(xs)` — lista invertida (extrato do mais recente para o mais antigo);

(d) `meu_map(f, xs)` — equivalente a `list(map(f, xs))`.

Questão 11 — Rendimento do CDB por recursão e por reduce

Dada uma lista `cdi` com as taxas mensais do CDI em fração e um percentual `p` do CDI pago pelo CDB (ex.: 1,10 para 110% do CDI), o rendimento acumulado do período é

$$R = \prod_{k=1}^n (1 + p \cdot c_k) - 1.$$

- (a) Implemente `rendimento_rec(cdi, p)` **por recursão estrutural** (cabeça/cauda), sem laços e sem `reduce`.
 - (b) Implemente `rendimento_red(cdi, p)` em **uma única expressão** com `reduce`.
 - (c) Verifique que ambas concordam (use `math.isclose`) para listas vazias, unitárias e de tamanho ≥ 3 .
 - (d) Justifique o caso-base de (a) e por que `rendimento_rec([], p)` deve retornar 0.
-

Questão 12 — Achatar o extrato agrupado

O app agrupa os lançamentos da conta por mês e por semana, produzindo listas **aninhadas** em profundidade arbitrária (créditos positivos, débitos negativos), por exemplo:

$$[[350,0; [-42,9; -120,0]]; []; [[1500,0]]; -89,9].$$

- (a) Implemente `achatar(xs)` por **recursão estrutural** (a cabeça pode ser uma lista). Use `isinstance(x, list)` para distinguir.
 - (b) Usando `achatar + filter + reduce` em uma única expressão, defina `total_gastos(xs)` que retorna a soma, em **valor absoluto**, apenas dos **débitos** (lançamentos estritamente negativos). No exemplo acima, o resultado é 252,8.
-

Questão 13 — Prazo para quitar um empréstimo (recursão)

Um cliente toma um empréstimo de valor D à taxa mensal i (fração) e paga uma parcela fixa P por mês. A cada mês, o saldo devedor é atualizado por

$$saldo_{k+1} = saldo_k \cdot (1 + i) - P, \quad saldo_0 = D.$$

O **prazo de quitação** é o menor $n \in \mathbb{N}$ tal que $saldo_n \leq 0$.

- (a) Defina, **por recursão** (sem laços, sem `math.log`, sem `**`), a função `meses_para_quitar(d, i, p)`. *Dica:* use uma auxiliar recursiva que mantém o saldo devedor e o número de meses já pagos.
 - (b) Prove informalmente a terminação quando $P > D \cdot i$. O que acontece se $P \leq D \cdot i$? Proponha um tratamento para esse caso.
 - (c) Calcule, à mão, `meses_para_quitar(300, 0,10, 120)` e `meses_para_quitar(1000, 0, 250)`, mostrando as chamadas recursivas até o caso-base.
-

Questão 14 — Ranking de fundos de investimento

Uma plataforma de investimentos recebe uma lista `fundos` de tuplas (`nome`, `[rentab_mensal_12m]`), isto é, cada fundo tem seu vetor com 12 rentabilidades mensais (em fração, podendo ser negativas).

- (a) Usando `map + reduce`, defina `rentabilidade_anual(fundos)` que devolve a lista de tuplas (`nome`, `rentab_acumulada`).
 - (b) Usando `filter`, defina `batem_benchmark(fundos, bench)` que retorna apenas os fundos cuja rentabilidade acumulada seja $\geq$ `bench`.
 - (c) Usando `reduce` (sem `max`, sem `sorted`), defina `melhor_fundo(fundos)` que retorna a tupla (`nome`, `rentab`) com **maior** rentabilidade acumulada.
 - (d) Combine (a), (b) e (c) em uma função `relatorio_fundos(fundos, bench)` que devolve um dicionário com chaves `"batem"`, `"melhor"` e `"qtd"`.
-

Questão 15 — Simulação de saldo de uma conta remunerada (recursão + funcional)

Uma conta digital rende à taxa mensal r sobre o saldo e, a cada mês t , registra uma movimentação líquida m_t (saques positivos, aportes negativos):

$$saldo_{t+1} = saldo_t \cdot (1 + r) - m_t,$$

onde `movs` = `[m0, m1, ..., mT-1]` é uma lista de tamanho T .

- (a) Defina **por recursão estrutural** sobre a lista `movs`, a função `trajetoria(saldo0, r, movs)` que devolve a lista $[saldo_0, saldo_1, \dots, saldo_T]$. Não use laços nem mutação. Não use `reduce`.
- (b) Reescreva (a) usando `reduce`, devolvendo a mesma lista. *Dica:* o acumulador é a lista de saldos já calculados; combine com a movimentação corrente.
- (c) Usando o resultado de (a) ou (b) e `map/filter/reduce`, defina:
- `saldo_medio(...)` — média dos saldos;
 - `meses_de_queda(...)` — quantos meses tiveram $saldo_{t+1} < saldo_t$.
- (d) Com $saldo_0 = 1000$, $r = 0,01$, $movs = [200; -500; 300; 100]$, calcule à mão a trajetória completa e verifique que sua função (a) produz o mesmo resultado.
- (e) Justifique formalmente a terminação da recursão em (a).
-